WorkNest: 200 Technical Questions

100 frontend and 100 backend questions people may ask about WorkNest, each with a short answer based on the project's real code. Use them to prepare for presentations and interviews.

Part A: Frontend (100 questions)

Basics and tooling

1. What is the frontend built with?
React 19 and TypeScript, bundled by Vite, styled with Tailwind CSS 4.

2. Why React?
It builds the UI from reusable components and re-renders only what changes when data changes.

3. Why TypeScript instead of plain JavaScript?
Types catch mistakes such as a missing field or a wrong prop before the code runs, and make editors autocomplete reliably.

4. What does Vite do?
It runs a fast development server with instant reload, and bundles the app into small static files for production.

5. What port does the dev server use?
`5173` by default (`http://localhost:5173`).

6. What command starts the frontend?
`npm run dev`. `npm run build` type-checks with `tsc -b` and then builds into `dist/`.

7. What is Tailwind CSS?
A utility-first CSS framework: styles are small classes like `rounded-full` written directly on elements.

8. Why Tailwind over separate CSS files?
Styles live next to the markup, so components are self-contained and there are no class-name clashes.

9. What does the tsc -b step in the build do?
It type-checks the whole project; the build fails if there is a type error, so broken code never ships.

10. What is ESLint used for?
Finding bugs and bad patterns, such as missing hook dependencies. Run it with `npm run lint`.

11. What is package.json?
The list of libraries the app depends on and the scripts that run it.

12. Difference between dependencies and devDependencies?
Dependencies are needed by the running app (React, axios); devDependencies only while developing (TypeScript, ESLint, Vite).

13. Which fonts are used?
Bricolage Grotesque for headings and Inter for body text, loaded from Google Fonts in `index.html`.

14. Where are images kept?
In `public/images`; files in `public/` are served as-is from the site root.

15. How was the favicon made?
The flower icon was cropped from the logo into `favicon.png`, `favicon.ico` and an Apple touch icon, linked in `index.html`.

React concepts

16. What is a component?
A function that returns JSX, such as `JobCard`, `Navbar` or `StatusPill`.

17. What is JSX?
HTML-like syntax inside TypeScript that React turns into real page elements.

18. What are props?
Inputs passed to a component, for example `<JobCard job={job} showSaveButton={false} />`.

19. What is state?
Data a component remembers between renders, created with `useState`: form fields, filters, loading flags.

20. What does useEffect do in this app?
Runs code after rendering, mainly to load data from the API when a page opens.

21. Why do some effects use a 'cancelled' flag?
If the user leaves the page before the request finishes, the flag stops the app from updating state on a page that's gone.

22. What is useMemo used for?
Remembering filtered lists (Candidates, Clients, Jobs, Pipeline) so they are only recalculated when their inputs change.

23. What is useRef used for?
Holding a reference to a DOM element, such as the navbar account menu, to detect clicks outside it.

24. What is a controlled input?
An input whose value comes from React state and updates through `onChange`; all forms here work that way.

25. Why does every list item have a key?
Keys let React track which item is which, so lists update correctly and efficiently.

26. What is conditional rendering?
Showing different UI depending on state, such as loading text, an error, an empty message, or the real list.

27. What is a custom hook here?
`useAuth()` and `useSavedJobs()` wrap React Context so any component can read who is logged in or which jobs are saved.

28. What is lifting state up?
Keeping state in a parent so several children share it, like the Apply page holding the whole application draft for all 4 steps.

29. How does the admin layout share data with its pages?
It loads the dashboard once and passes it through React Router's `Outlet` context, so Overview and Notifications reuse it.

30. Why is the Jobs page search synced during render, not in an effect?
React's recommended pattern for resetting state when a prop or URL changes; it avoids an extra render and satisfies the lint rules.

Routing and navigation

31. Which router library is used?
React Router v7, with all routes declared in `src/App.tsx`.

32. What is a single-page application?
One HTML page loads once; React swaps the content when the URL changes, so pages don't reload.

33. How are admin pages nested?
Under `/admin`, which renders `AdminLayout` with the sidebar; child pages appear in its `<Outlet />`.

34. What happens on an unknown URL?
A catch-all route shows a 404 page; unknown `/admin/...` paths redirect to the dashboard.

35. What is a dynamic route?
A route with a parameter, like `/job-details/:jobId`; the page reads it with `useParams()`.

36. How does the search keep its terms in the URL?
Find Jobs reads and writes `?q=` and `?location=` with `useSearchParams`, so searches can be bookmarked and shared.

37. What does ScrollToTop do?
Scrolls to the top on every page change, or to a section when the URL has a hash such as `/#how-it-works`.

38. What is a protected route?
`ProtectedRoute` sends logged-out users to Log in and non-staff away from `/admin`.

39. How does the app return you to where you were after logging in?
The redirect to Log in passes `state.from`; after login the app navigates back to it.

40. Why does Vercel need vercel.json?
It rewrites every URL to `index.html`, so refreshing on `/find-jobs` loads the app instead of a 404.

41. What's the difference between Link and NavLink?
`NavLink` knows when its route is active, which is how the navbar highlights the current page.

42. How do the footer's company links work?
They point to real pages: `/about`, `/for-companies`, `/contact`, `/terms` and `/privacy`.

43. How does the job menu in admin open the pipeline for one job?
It links to `/admin/pipeline?job=<id>`, and the Pipeline reads that parameter to preselect the role.

Talking to the API

44. Which library makes HTTP requests?
axios, through one shared client in `src/lib/apiClient.ts`.

45. How does the frontend know the API address?
From `VITE_API_URL` in `.env`, defaulting to `http://localhost:5000/api`.

46. Why must frontend env variables start with VITE_?
Vite only exposes variables with that prefix to the browser, which stops server secrets leaking into the page.

47. How is the login token attached to requests?
An axios request interceptor reads it from localStorage and adds `Authorization: Bearer <token>`.

48. Where are API calls organised?
In `src/api/`, one file per area (jobs, auth, profile, applications...), each function making one call.

49. How are API errors shown to users?
`extractError()` pulls the server's message, and pages show it in a toast, for example "Invalid email or password".

50. What is a toast?
A small pop-up message. WorkNest uses react-toastify through a helper in `src/lib/toast.tsx`.

51. How are files uploaded?
With `FormData`: the file is appended under a field name (`cv` or `logo`) and posted with axios.

52. Can guests upload a CV when applying?
Yes. The Apply page uses `POST /api/applications/cv`, which doesn't need a login and never touches anyone's profile.

53. Why are some requests run in parallel?
`Promise.all` starts several independent requests together, as the Apply page does for the job, prefill and already-applied check.

54. Is job search done on the frontend or backend?
Frontend: the API sends all open jobs once and the browser filters them instantly as you type.

55. When would you move search to the server?
With thousands of jobs: then the server would filter and paginate so the browser doesn't download everything.

Authentication on the frontend

56. Where is the login state kept?
In `AuthContext`, which shares the current user with every page.

57. Where is the token stored?
In localStorage under `worknest_token`.

58. What happens when the app first loads?
It calls `GET /api/auth/me` to check the stored token and load the user.

59. What if that check fails because of a network blip?
It retries up to 3 times with a short wait; only a 401 logs the user out, so a slow server doesn't sign people out.

60. How does logout work?
The token is removed from localStorage and the user is cleared from context.

61. How does Continue with Google work?
The `@react-oauth/google` popup returns an access token, which the frontend sends to the API to verify.

62. How is the 6-digit code entered?
With `OtpInput`, six boxes that auto-advance, accept paste, and handle backspace.

63. How do forms show validation errors?
Each field gets its own red message on submit, cleared as soon as you edit that field.

64. Is hiding admin pages in the browser real security?
No. It only improves the experience; the server rejects non-staff requests on its own.

65. What does SavedJobsContext do?
Loads the ids of saved jobs after login so every bookmark icon fills in instantly, and toggles saves through the API.

Pages and features

66. How does the Apply wizard work?
Four steps (Your details, CV and links, Questions, Review), each validated before you continue.

67. How does the application pre-fill?
For logged-in users it calls `/applications/prefill` and copies name, email, phone, CV and links from the profile.

68. What stops someone applying twice?
The Apply page checks first and sends you to Applications; the server also refuses duplicates.

69. How is the profile completion score calculated?
Five sections (basics, experience, skills, CV and links, preferences), each worth 20%.

70. Why did uploading a CV once reset the score?
The upload replaced the whole form with the saved profile; now only the CV fields are updated, so unsaved edits stay.

71. How does the home page show jobs?
It loads real open jobs from the API and shows up to 6, with placeholder cards while loading.

72. How does Find Jobs sort?
Newest uses days since posting, Closing soon uses days left, and Highest salary uses the top of the range.

73. How is 'closes in X days' shown?
The API sends a number of days; `JobCard` shows 'Closes today' at 0.

74. What's on the Applications page?
Your applications with their stage, saved jobs, and notifications you can mark as read.

75. How does the admin Candidates page stay readable on phones?
It shows a table from 1280px wide and switches to cards below that.

76. How does the admin Overview export work?
It builds a CSV from the dashboard data in the browser and downloads it with a Blob, with no server call.

77. How do bulk actions work in the Pipeline?
Tick candidates, choose a stage, press Apply: the app updates each one in parallel and reports how many succeeded.

78. What does the admin search box do?
Pressing Enter opens the Jobs page filtered by your text through `?q=`.

79. How does the Contact form send messages?
It opens your email app with the subject and message filled in, using a `mailto:` link.

80. How does copying a job link work?
With the browser Clipboard API, with a fallback message showing the link if copying is blocked.

UI, accessibility and quality

81. Is the site responsive?
Yes. Layouts use Tailwind breakpoints (sm, md, lg, xl), and the navbar becomes a hamburger menu on small screens.

82. How does the account menu animate?
It stays mounted and transitions opacity, scale and position; a hidden menu can't be clicked or tabbed into.

83. Do animations respect reduced-motion settings?
Yes. The account menu uses `motion-reduce:transition-none`.

84. How are dropdowns closed?
By clicking outside or pressing Escape, using document listeners that are removed on cleanup.

85. What accessibility features exist?
Labels on icon buttons (`aria-label`), `aria-expanded` on menus, `role="menu"`, and real buttons and links instead of clickable divs.

86. How are loading states handled?
Every data page shows a loading message or skeleton cards, then the data, an empty message or an error.

87. How are avatars shown without photos?
Initials in a coloured circle, with the colour picked consistently from the name.

88. Why is there no state-management library like Redux?
The app's shared state is small (the user and saved jobs), so React Context is enough.

89. What would you test first if you added tests?
The Apply wizard's validation, the profile completion score, and the auth context's retry logic.

90. How would you improve performance?
Code-split admin pages with lazy loading; the build already warns that the main bundle is over 500 KB.

Project structure and extras

91. What does main.tsx do?
It starts the app and wraps it in the Google sign-in provider, the router, the auth and saved-jobs providers, and the toast container.

92. What is React StrictMode?
A development-only wrapper that runs some code twice to expose bugs early; it does nothing in production.

93. Why do some imports say 'import type'?
The `verbatimModuleSyntax` setting requires it; type-only imports are removed completely from the built JavaScript.

94. What does noUnusedLocals do?
Makes the build fail if a variable is declared but never used, keeping the code clean.

95. How are dates shown as '2 days ago'?
With small helpers in `src/utils/` that turn a date into relative text.

96. What does the Stepper component show?
A row of steps for an application's progress, used on application cards and the success screen.

97. How do the legal pages share a layout?
They all use `InfoPage` and `InfoSection`, so the Terms, Privacy, About and Contact pages look consistent.

98. How are external links opened safely?
With `target="_blank"` and `rel="noopener noreferrer"`, so the new tab can't control the WorkNest tab.

99. How does the CSV export handle commas in names?
Any cell containing a comma, quote or line break is wrapped in quotes, with quotes doubled, so the file opens correctly in Excel.

100. What does the 404 page offer?

A clear message and buttons back to the home page and to Find Jobs.

Part B: Backend (100 questions)

Basics and setup

1. What is the backend built with?
Node.js and Express 5, written in TypeScript, storing data in MongoDB through Mongoose 9.

2. What is Node.js?
A runtime that lets JavaScript run on a server.

3. What is Express?
A web framework for Node.js that handles routes, middleware and responses.

4. Why Express 5?
It is the current major version and passes errors from async route handlers to Express automatically.

5. What port does the server use?
The `PORT` environment variable, or 5000 by default.

6. How do you run it in development?
`npm run dev`, which uses `tsx watch` to restart on every save.

7. How is it built for production?
`npm run build` compiles TypeScript into `dist/`; `npm start` runs `node dist/server.js`.

8. What are the Render build and start commands?
Build: `npm install && npm run build`. Start: `npm start`.

9. Why connect to the database before listening?
So the server never accepts requests it can't answer; if the connection fails, the process exits.

10. What is the health check?
`GET /api/health` returns `{"status":"ok"}` without touching the database.

11. What is dotenv for?
It loads settings and secrets from `.env` into `process.env`.

12. Which environment variables are needed?
`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `GOOGLE_CLIENT_ID`, `CLIENT_ORIGIN`, `PORT`, plus Brevo and Cloudinary keys.

13. Why is .env not committed?
It contains secrets; it's in `.gitignore` and `.env.example` lists the names instead.

14. What module system does the server use?
CommonJS, set in `package.json` and `tsconfig.json`.

15. What is tsx?
A tool that runs TypeScript directly without compiling first, used for development and scripts.

Architecture and REST

16. What is a REST API?
URLs represent resources and HTTP methods are the actions: GET reads, POST creates, PATCH updates, DELETE removes.

17. How is the code organised?
routes (URLs), middleware (checks), controllers (logic), models (data), services (email), config (database, Cloudinary).

18. What is a route?
A URL and method linked to its handler, for example `GET /api/jobs` in `routes/jobRoutes.ts`.

19. What is a controller?
The function that does the work: reads the request, queries the database, and sends the response.

20. What is middleware?
A function that runs before the controller, such as checking the token or reading an uploaded file.

21. What does express.json() do?
Parses JSON request bodies into `req.body`.

22. What is CORS and why is it needed?
A browser rule blocking calls to other addresses; the `cors` middleware allows the frontend to call the API.

23. Why is CORS set to allow any origin?
Auth uses Bearer tokens, not cookies, so a wildcard origin can't be used to send a user's credentials.

24. How many endpoints are there?
About 40, grouped under `/api/auth`, `/api/jobs`, `/api/profile`, `/api/applications`, `/api/admin/...` and more.

25. Why are admin routes under /api/admin?
To keep staff-only endpoints separate, so their protection is applied in one place.

26. Why must POST /api/applications/cv be declared before /:jobId?
Express matches in order; otherwise the word 'cv' would be treated as a job id.

27. What does the API return on errors?
JSON like `{"message": "..."}` with a matching status code, which the frontend shows to the user.

28. Which status codes are used?
200 OK, 201 Created, 400 bad input, 401 not logged in, 403 not allowed, 404 not found, 429 too many requests, 500 server error.

29. Why do controllers wrap logic in try/catch?
So unexpected failures are logged and return a clean 500 message instead of crashing or leaking details.

30. How are response shapes kept consistent?
Helper functions like `toClientJob` turn database records into exactly what the frontend expects.

Database and Mongoose

31. Which database is used?
MongoDB, hosted on MongoDB Atlas.

32. Why MongoDB instead of SQL?
Profiles and jobs vary in shape (lists of skills, screening answers), which suits documents; Mongoose still enforces a schema.

33. What is Mongoose?
A library that defines schemas, validates data, and provides models to query MongoDB.

34. Which models exist?
Six: User, Company, Job, Application, SavedJob, Notification.

35. How are models related?
By references: a Job stores its Company's id; an Application stores its Job and User ids.

36. What does populate() do?
Replaces a stored id with the referenced record, like a job's company name and logo.

37. What does lean() do?
Returns plain objects instead of Mongoose documents: faster for reading, but schema defaults aren't applied.

38. Give an example where lean() caused a bug.
Older jobs had no `screeningQuestions` field, so lean() returned undefined and the Apply page crashed; the API now defaults it to an empty list.

39. What are timestamps?
Automatic `createdAt` and `updatedAt` fields on every record.

40. What is an index and where is one used?
A lookup structure that speeds queries; for example Job has an index on status and creation date.

41. How are duplicate applications prevented?
A unique index on `{job, applicant}` with a partial filter, so it only applies when there is an account.

42. Why the partial filter?
Guests have no applicant id; without it, only one guest could ever apply to each job.

43. How are saved jobs kept unique?
A unique index on `{user, job}` in SavedJob.

44. How are duplicate notifications avoided?
Each has a `dedupeKey` with a unique index per user, so the same alert can't be created twice.

45. What job statuses exist?
draft, open, closed and archived.

46. What application stages exist?
submitted, review, shortlisted, interview, offer, hired, and rejected.

47. How is stage history kept?
Every change is pushed onto `statusHistory` with the date, so candidates see a timeline.

48. What is in the scorecard?
Four scores from 0 to 5: skills match, experience, communication, and portfolio or work.

49. What does the seed script do?
`npm run seed` wipes and refills jobs and companies; `npm run seed-candidates` adds 14 demo candidates.

50. How do you make someone an admin?
`npm run make-admin -- their@email.com`; it updates that user's role in the database.

Authentication and security

51. How does registration work?
The server creates the user, hashes the password, and emails a 6-digit code.

52. How are passwords stored?
As bcrypt hashes with cost 10; the real password is never saved.

53. What does select: false do?
Hides fields such as password and code hashes from every query unless explicitly requested.

54. How is the email code generated?
With `crypto.randomInt`, six digits, stored only as a bcrypt hash.

55. How long is a code valid, and how many tries?
10 minutes, and 5 wrong attempts before a new code is required.

56. Why is the code hashed?
So a leaked database doesn't reveal codes that could be used to take over accounts.

57. Why must a reset code not verify an email?
Each code records its purpose, so one issued for password reset can't be used for verification, or the other way round.

58. What is a JWT?
A signed token proving who the user is; WorkNest's contains the user id and role.

59. How long does a token last?
7 days by default, set by `JWT_EXPIRES_IN`.

60. What does the protect middleware do?
Verifies the Bearer token and sets `req.userId` and `req.userRole`, or returns 401.

61. What does authorize do?
Checks the role (admin or recruiter) and returns 403 if it doesn't match.

62. What does identify do?
Reads a token if present but never blocks; used where guests are allowed, such as applying.

63. What's the difference between 401 and 403?
401: we don't know who you are. 403: we know who you are, but you're not allowed.

64. How is Google sign-in verified?
The server calls Google's tokeninfo endpoint and checks the token was issued for WorkNest's own Client ID.

65. Why check the token's audience?
Otherwise a token issued to another app could be replayed to log into WorkNest.

66. How do you stop users promoting themselves to admin?
Profile updates accept only a whitelist of fields; role, email and verification can't be changed there.

67. How are recruiters' private notes kept private?
The candidate-facing API never includes scorecards or internal notes.

68. What is rate limiting used for?
Guest CV uploads are limited to 10 per hour per IP with express-rate-limit, returning 429 after that.

69. Why does rate limiting need care on Render?
Render uses a proxy, so Express must trust it to see real visitor IPs, or all users would share one limit.

70. What would you add for more security?
Login rate limiting, Helmet security headers, input validation with a schema library, and refresh tokens.

Files and email

71. How are uploads received?
Multer reads the multipart form into memory, not onto disk.

72. Why memory storage?
The file goes straight to Cloudinary, so nothing is left on the server's disk.

73. What limits apply to CVs?
PDF, DOC or DOCX only, up to 5 MB.

74. What limits apply to logos?
PNG, JPEG, WEBP or SVG, up to 2 MB.

75. How does a file reach Cloudinary?
The buffer is streamed with `upload_stream`; CVs as raw files in `worknest/cvs`, logos as images in `worknest/logos`.

76. What is stored in the database for a file?
Only the Cloudinary link and the original file name.

77. How are upload errors handled?
Upload middleware turns Multer errors into clear 400 messages like 'That file is too large'; both CV routes share `handleCvUpload`.

78. Which service sends email?
Brevo, through its official Node SDK.

79. Which emails are sent?
Verification code, password reset code, welcome, application received, and status change.

80. What happens if email sending fails?
The error is logged and the request still succeeds; in development the code is printed in the terminal.

Business logic and data

81. Which jobs appear publicly?
Only open jobs whose closing date hasn't passed.

82. How is 'closes in X days' calculated?
Days until the closing date, rounded up, never below zero.

83. Are closed jobs hidden from saved jobs?
Yes. Saved jobs use the same open-and-not-expired rule as Find Jobs.

84. Can someone apply to a closed job?
No. The server checks the status and closing date before accepting an application.

85. How does the dashboard load so many numbers quickly?
It runs about a dozen independent queries at once with `Promise.all`.

86. What is an aggregation pipeline?
A MongoDB query in stages (match, group, sort) used for counts like applications per status and per candidate.

87. How is a candidate's status decided?
Placed if any application is hired; Employed if they set openToWork to false; otherwise Open to work.

88. Why do candidates sometimes show application data?
If the profile is empty, the Candidates page falls back to the location and experience from their latest application.

89. How are client counts calculated?
Live: open roles from Job and placements from hired Applications, not stored numbers.

90. What would you build next on the backend?
Server-side search and pagination, job editing and closing endpoints, automated tests, and scheduled jobs for reminders.

Code structure and deployment

91. What does server.ts do?
Creates the Express app, adds CORS and JSON parsing, mounts every route group, connects to MongoDB, then starts listening.

92. What is express.d.ts for?
It tells TypeScript that requests can carry `userId` and `userRole`, which the auth middleware adds.

93. Why are enums kept in types/enums.ts?
The same `as const` lists serve as Mongoose's allowed values and as TypeScript types, so they can never disagree.

94. What is a slug and how is it made?
A URL-friendly name like `frontend-engineer`, made from the title and given a suffix if it's already taken.

95. What does toClientJob compute?
Days left to apply, days since posting, and the company's name and logo, in the shape the frontend expects.

96. How does the Candidates endpoint count applications?
One aggregation groups all applications by applicant, counting the total and how many were hired.

97. How are 'this week' and 'this month' figures calculated?
By counting records whose `createdAt` is after a date 7 or 30 days ago.

98. What can the notifications endpoints do?
List your notifications, mark one as read, or mark them all as read.

99. Where is everything deployed?
The frontend on Vercel, the API as a Render web service, and the database on MongoDB Atlas.

100. How would you add automated tests?
With a test runner like Vitest plus Supertest, against an in-memory MongoDB, starting with auth, applying and admin access.